

CyberAudit Platform

Railway Deployment Summary

Résumé des Déploiements Railway

Project / Projet	CyberAudit
Platform / Plateforme	Railway.app
Date	June-July 2026 / Juin-Juillet 2026
Author / Auteur	Tommy Jouhans and James ROUSSEL
Services	PostgreSQL · Redis · Backend · Portfolio

Architecture Overview

Vue d'ensemble de l'architecture

The CyberAudit platform is deployed on **Railway.app** across 4 independent services that communicate internally. Each service is versioned, auto-deployed from GitHub, and monitored in real time.

*La plateforme CyberAudit est déployée sur **Railway.app** en 4 services indépendants qui communiquent en interne. Chaque service est versionné, déployé automatiquement depuis GitHub, et surveillé en temps réel.*

Service	Type	URL / Access	Status
PostgreSQL	Managed Database	Internal only / Interne uniquement	✓ Active
Redis	Cache / Message Broker	Internal only / Interne uniquement	✓ Active
Backend	Django REST API	cyberaudit-backend.up.railway.app	✓ Active
Portfolio	React Frontend	Not publicly exposed	✓ Active

1. PostgreSQL : Managed Database

1. PostgreSQL : Base de données gérée

What is it? / Qu'est-ce que c'est ?

PostgreSQL is the **main relational database** of the CyberAudit platform. It stores all structured data: users, audit requests, packs, training modules, notifications and reports.

*PostgreSQL est la **base de données relationnelle principale** de la plateforme CyberAudit. Elle stocke toutes les données structurées : utilisateurs, demandes d'audit, packs, modules de formation, notifications et rapports.*

Configuration

Image	Railway managed PostgreSQL (latest)
Accessibility / Accès	Internal network only, not exposed publicly Réseau interne uniquement, non exposé publiquement
Connection	\$DATABASE_URL environment variable Variable d'environnement \$DATABASE_URL
Driver / Pilote	psycopg2 (via Django ORM)
Auto-migrate	python manage.py migrate runs on each deploy S'exécute automatiquement à chaque déploiement

Main tables / Tables principales

accounts_user	Custom user model : email, role (admin/client), active status Modèle utilisateur, email, rôle, statut actif
audits_auditrequest	Audit requests with UUID PK : status lifecycle Demandes d'audit avec UUID, cycle de vie des statuts
packs_pack	Subscription packs available to clients Packs d'abonnement disponibles pour les clients
training_module	Training modules with user progress tracking Modules de formation avec suivi de la progression
notifications_notification	In-app notifications per user Notifications in-app par utilisateur
reports_report	Generated PDF report metadata Métadonnées des rapports PDF générés

Role in the architecture / Rôle dans l'architecture

The Backend (Django) is the only service that reads and writes to PostgreSQL. Redis and Portfolio have no direct database access.

Le Backend (Django) est le seul service qui lit et écrit dans PostgreSQL. Redis et Portfolio n'ont pas d'accès direct à la base de données.

2. Redis : Cache & Message Broker

2. Redis : Cache et courtier de messages

What is it? / Qu'est-ce que c'est ?

Redis is an in-memory data store used as a **message broker and task queue** for Celery. It allows the Django backend to execute long-running tasks (PDF generation, email sending) asynchronously without blocking HTTP responses.

*Redis est un store en mémoire utilisé comme **courtier de messages et file d'attente de tâches** pour Celery. Il permet au backend Django d'exécuter des tâches longues (génération de PDF, envoi d'emails) de façon asynchrone sans bloquer les réponses HTTP.*

Configuration

Image	Railway managed Redis (latest)
Accessibility / Accès	Internal network only — not exposed publicly Réseau interne uniquement — non exposé publiquement
Connection	\$REDIS_URL environment variable Variable d'environnement \$REDIS_URL
Celery Broker	CELERY_BROKER_URL = \$REDIS_URL
Celery Backend	CELERY_RESULT_BACKEND = \$REDIS_URL

How Celery + Redis work / Fonctionnement Celery + Redis

1. Client triggers task	POST /api/audits/{uuid}/generate-report/ → Django queues a Celery task
EN → Django responds	Immediately returns HTTP 202 Accepted (async, non-blocking)
2. Redis stores the job	The task is placed in the Redis queue La tâche est placée dans la file d'attente Redis
3. Celery worker picks up	The worker reads the task, generates the PDF with WeasyPrint Le worker lit la tâche, génère le PDF avec WeasyPrint
4. Result saved	Report saved to database; email notification sent to client Rapport sauvegardé en base de données ; email envoyé au client

Why Redis? / Pourquoi Redis ?

Without Redis, PDF generation would run synchronously inside the HTTP request, causing the request to time out on large reports. Redis decouples the heavy work from the API layer.

Sans Redis, la génération de PDF s'exécuterait de manière synchrone dans la requête HTTP, provoquant des timeouts pour les gros rapports. Redis découple le travail lourd de la couche API.

3. Backend : Django REST API

3. Backend : API REST Django

What is it? / Qu'est-ce que c'est ?

The Backend is the **core of the CyberAudit platform**. It exposes a REST API built with Django REST Framework, handles authentication (JWT), business logic, async tasks, and PDF report generation.

Le Backend est le cœur de la plateforme CyberAudit. Il expose une API REST construite avec Django REST Framework, gère l'authentification (JWT), la logique métier, les tâches asynchrones et la génération de rapports PDF.

Technical stack / Stack technique

Language	Python 3.11+
Framework	Django 4.x + Django REST Framework
Authentication / Authentification	SimpleJWT — access token (15 min) + refresh token (7 days)
Database ORM	Django ORM → PostgreSQL
Async tasks / Tâches async	Celery + Redis → WeasyPrint (PDF)
Permissions / Permissions	RBAC: admin role vs client role
Public URL	https://cyberaudit-backend.up.railway.app
API prefix / Préfixe API	/api/ — all endpoints start with /api/ Tous les endpoints commencent par /api/

Main API endpoints / Principaux endpoints API

Method	Endpoint	Access / Accès	Description (EN / FR)
POST	/api/auth/register/	Public	Create account / Créer un compte
POST	/api/auth/login/	Public	Get JWT tokens / Obtenir tokens JWT
GET	/api/auth/me/	Auth	Get profile / Voir profil
PATCH	/api/auth/me/	Auth	Update profile / Modifier profil
POST	/api/auth/token/refresh/	Auth	Renew access token / Renouveler le token

GET	/api/packs/	Public	List packs / Lister les packs
POST	/api/audits/	Client	Submit audit request / Soumettre une demande d'audit
GET	/api/audits/	Auth	List audits (role-filtered) / Lister les audits
PATCH	/api/audits/{uuid}/	Admin	Update audit status / Modifier le statut d'audit
DELETE	/api/audits/{uuid}/	Auth	Soft delete (archived) / Suppression douce
GET	/api/training/modules/	Auth	List training modules / Lister les modules
POST	/api/training/modules/{id}/start/	Auth	Start module / Démarrer un module
POST	/api/training/modules/{id}/complete/	Auth	Complete module / Terminer un module
GET	/api/notifications/me/	Auth	Get notifications / Voir notifications
POST	/api/audits/{uuid}/generate-report/	Admin	Queue PDF generation (202) / Lancer la génération PDF
GET	/api/audits/{uuid}/report/	Auth	Download PDF / Télécharger le PDF

Important rules / Règles importantes

Trailing slash /	All Django URLs require a trailing slash Tous les URLs Django nécessitent un slash final
JWT Authorization	Header: Authorization: Bearer <access_token> En-tête : Authorization: Bearer <access_token>
Token expiry / Expiration	access token: 15 minutes, refresh token: 7 days access token : 15 minutes, refresh token : 7 jours
RBAC	admin can see all audits; client sees only own admin voit tous les audits, client voit uniquement les siens
Soft delete / Suppression douce	DELETE on audit sets status=archived, record is kept DELETE sur un audit passe le statut à archived, l'enregistrement est conservé

4. Portfolio — React Frontend

4. Portfolio — Frontend React

What is it? / Qu'est-ce que c'est ?

The Portfolio service hosts the **React frontend application** of CyberAudit. It provides the user interface used by clients to submit audits, follow their progress, and access training modules.

*Le service Portfolio héberge l'**application frontend React** de CyberAudit. Il fournit l'interface utilisateur utilisée par les clients pour soumettre des audits, suivre leur progression et accéder aux modules de formation.*

Configuration

Framework	React (Vite or Create React App)
Railway service	portfolio
Accessibility / Accès	Not publicly exposed via Railway domain Non exposé publiquement via le domaine Railway
API target / Cible API	Points to Backend URL in environment variables Pointe vers l'URL du Backend dans les variables d'environnement
Auto-deploy	Triggered on push to main branch (GitHub) Déclenché lors d'un push sur la branche main (GitHub)
Build command	npm run build
Start command	serve -s dist or similar static server Serveur statique type serve -s dist

Key note / Note importante

Hardcoded data vs API data

Some pages in the frontend display hardcoded (static) data, especially the training modules section. This is why the frontend shows content even when the database is empty. Always use the Postman API to verify real database content.

Certaines pages du frontend affichent des données codées en dur (statiques), notamment la section modules de formation. C'est pourquoi le frontend affiche du contenu même quand la base de données est vide. Utilisez toujours l'API Postman pour vérifier le vrai contenu de la base de données.

Global Summary

Résumé Global

Service	Type	Exposed / Exposé	Role EN / FR	Dependencies
PostgreSQL	Database	✗ Internal only	Stores all data Stocke toutes les données	None / Aucune
Redis	Cache / Broker	✗ Internal only	Async task queue File de tâches async	None / Aucune
Backend	Django REST API	✓ Public HTTPS	API, auth, business logic API, auth, logique métier	PostgreSQL Redis Celery worker
Portfolio	React App	⚠ Not exposed	User interface Interface utilisateur	Backend API

Communication flow / Flux de communication: Portfolio → Backend (REST API via HTTPS) → PostgreSQL (ORM) + Redis (Celery tasks)